

```
#Task 1 - Customer Data Cleaning
#Prompt:Generate a Python script to clean customer data by handling missing values, removing duplicates, standardizing text
import pandas as pd
from sklearn.preprocessing import LabelEncoder
# Load dataset
data = pd.read_csv("/content/shopping_trends (1).csv")
# Display original data
print("Original Data:\n")
print(data.head())
# 1. Handle Missing Values
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Purchase Amount (USD)'].fillna(data['Purchase Amount (USD)'].median(), inplace=True)
data['Location'].fillna(data['Location'].mode()[0], inplace=True)

# 2. Remove Duplicate Records
data.drop_duplicates(inplace=True)

# 3. Standardize Text Values
data['Location'] = data['Location'].str.lower()

# 4. Encode Categorical Variables
le = LabelEncoder()
data['Gender'] = le.fit_transform(data['Gender'])
data['Location'] = le.fit_transform(data['Location'])
print("\nCleaned Data:\n")
print(data.head())
# Optional: Check dataset info
print("\nDataset Info:\n")
print(data.info())
```

	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	14	Venmo	Fortnightly

```
data['Location'].fillna(data['Location'].mode()[0], inplace=True)
```

```
#Task 2 - Sales Data Preprocessing
#prompt:"Generate a Python script to preprocess sales data by converting dates, extracting time features, normalizing values
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
data = pd.read_csv("/content/sales_data.csv")

# Display original data
print("Original Data:\n")
print(data.head())

# -----
# 1. Convert Date to DateTime
# -----
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])

# -----
# 2. Create New Time Features
# -----
data['Month'] = data['Sale_Date'].dt.month
data['Year'] = data['Sale_Date'].dt.year
data['DayOfWeek'] = data['Sale_Date'].dt.dayofweek

# -----
# 3. Normalize Sales Column
# -----
scaler = MinMaxScaler()
data['Sales_Amount'] = scaler.fit_transform(data[['Sales_Amount']])

# -----
# 4. Detect & Remove Outliers (IQR)
# -----
Q1 = data['Sales_Amount'].quantile(0.25)
Q3 = data['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

data = data[(data['Sales_Amount'] >= Q1 - 1.5 * IQR) &
            (data['Sales_Amount'] <= Q3 + 1.5 * IQR)]

# -----
# Final Processed Data
# -----
print("\nProcessed Sales Data:\n")
print(data.head())

# Optional: dataset summary
print("\nDataset Info:\n")
print(data.info())
```

```
Payment_Method Sales_Channel Region_and_Sales_Rep
0          Cash          Online      North-Bob
1          Cash          Retail      West-Bob
2  Bank Transfer          Retail    South-David
3   Credit Card          Retail    South-Bob
4   Credit Card          Online    East-Charlie
```

Processed Sales Data:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
0	1052	2023-02-03	Bob	North	0.500950	18	
1	1093	2023-04-21	Bob	West	0.433202	17	
2	1015	2023-09-21	David	South	0.458201	30	
3	1072	2023-08-24	Bob	South	0.209105	39	
4	1061	2023-03-24	Charlie	East	0.369108	13	

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product_ID            1000 non-null   int64
1   Sale_Date             1000 non-null   datetime64[ns]
2   Sales_Rep             1000 non-null   object
3   Region                1000 non-null   object
4   Sales_Amount          1000 non-null   float64
5   Quantity_Sold         1000 non-null   int64
6   Product_Category      1000 non-null   object
7   Unit_Cost             1000 non-null   float64
8   Unit_Price            1000 non-null   float64
9   Customer_Type         1000 non-null   object
10  Discount              1000 non-null   float64
11  Payment_Method        1000 non-null   object
12  Sales_Channel         1000 non-null   object
13  Region_and_Sales_Rep  1000 non-null   object
14  Month                 1000 non-null   int32
15  Year                  1000 non-null   int32
16  DayOfWeek             1000 non-null   int32
dtypes: datetime64[ns](1), float64(4), int32(3), int64(2), object(7)
memory usage: 121.2+ KB
```

#Task 3 – Healthcare Data Preparation

#prompt:Generate a Python script to preprocess healthcare data by handling missing values, encoding categorical features, sc

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("/content/dirty_v3_path.csv")

# Display original data
print("Original Data:\n")
print(data.head())

# -----
# 1. Handle Missing Values
# -----
data['Blood Pressure'].fillna(data['Blood Pressure'].mean(), inplace=True)
data['Cholesterol'].fillna(data['Cholesterol'].mean(), inplace=True)

# -----
# 2. Encode Categorical Features
# -----
le = LabelEncoder()

for col in data.select_dtypes(include=['object']).columns:
    data[col] = le.fit_transform(data[col])

# -----
# 3. Scale Numerical Features
# -----
scaler = MinMaxScaler()

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

# -----
# 4. Split Dataset (Train/Test)
# -----
# If target column exists, use it
if 'target' in data.columns:
    X = data.drop('target', axis=1)
    y = data['target']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    print("\nTrain/Test Split Done")
    print("X_train shape:", X_train.shape)
    print("X_test shape:", X_test.shape)

else:
    print("\nPreprocessed Data:\n")
    print(data.head())

# Optional: dataset info
print("\nDataset Info:\n")
print(data.info())
```

Original Data:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	\
0	46.0	Male	Diabetes	137.04	135.27	28.90	
1	22.0	Male	Healthy	71.58	113.27	26.29	
2	50.0	NaN	Asthma	95.24	NaN	22.53	
3	57.0	NaN	Obesity	NaN	130.53	38.47	
4	66.0	Female	Hypertension	95.15	178.17	31.12	

	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	\
0	96.04	6	231.88	210.56	7.61	
1	97.54	2	165.57	129.41	4.91	
2	90.31	2	214.94	165.35	5.60	
3	96.60	5	197.71	182.13	6.92	
4	94.90	4	259.53	115.85	5.98	

	Smoking	Alcohol	Physical Activity	Diet Score	Family History	\
0	0	0	-0.20	3.54	0	
1	0	0	8.12	5.90	0	
2	0	0	5.01	4.65	1	
3	0	0	3.16	3.37	0	
4	0	1	3.56	3.40	0	

	Stress Level	Sleep Hours	random_notes	noise_col
0	5.07	6.05	lorem	-137.057211
1	5.87	7.72	ipsum	-11.230610
2	3.09	4.82	ipsum	98.331195
3	3.01	5.33	lorem	44.187175
4	6.38	6.64	lorem	44.831426

Preprocessed Data:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	\
0	0.455696	0.5	0.428571	0.391428	0.401144	0.431680	
1	0.151899	0.5	0.571429	0.171904	0.256540	0.378609	
2	0.506329	1.0	0.142857	0.251249	0.435226	0.302155	
3	0.594937	1.0	0.857143	NaN	0.369988	0.626271	
4	0.708861	0.0	0.714286	0.250947	0.683121	0.476820	

	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	\
0	0.670348	0.277778	0.518390	0.524877	0.476872	
1	0.705592	0.055556	0.265915	0.342102	0.179515	
2	0.535714	0.055556	0.453891	0.423050	0.255507	
3	0.683506	0.222222	0.388288	0.460844	0.400881	
4	0.643562	0.166667	0.623667	0.311561	0.297357	

	Smoking	Alcohol	Physical Activity	Diet Score	Family History	\
0	0.0	0.0	0.216283	0.383056	0.0	
1	0.0	0.0	0.733375	0.553946	0.0	
2	0.0	0.0	0.540087	0.463432	1.0	
3	0.0	0.0	0.425109	0.370746	0.0	
4	0.0	1.0	0.449969	0.372918	0.0	

	Stress Level	Sleep Hours	random_notes	noise_col
0	0.419788	0.509132	1.000000	0.312605
1	0.464505	0.699772	0.666667	0.455579
2	0.309111	0.368721	0.666667	0.580072
3	0.309111	0.368721	1.000000	0.580072
4	0.309111	0.368721	1.000000	0.580072

#Task 4 -- Employee Salary Data Preprocessing

#prompt:Generate a Python script to preprocess healthcare data by handling missing values, encoding categorical features, sc

import pandas as pd

from sklearn.preprocessing import LabelEncoder

Load dataset

data = pd.read_csv("/content/employee_salary_dataset.csv")

print("Original Data:\n", data.head())

1. Handle Missing Values

data.fillna(method='ffill', inplace=True)

2. Remove Salary Outliers (IQR)

salary_col = [col for col in data.columns if "salary" in col.lower()][0]

Q1 = data[salary_col].quantile(0.25)

Q3 = data[salary_col].quantile(0.75)

IQR = Q3 - Q1

data = data[(data[salary_col] >= Q1 - 1.5 * IQR) &

.....(data[salary_col] <= Q3 + 1.5 * IQR)]

```
#-----
# 3. Standardize Text Columns
#-----
for col in data.select_dtypes(include='object').columns:
    data[col] = data[col].str.lower()

#-----
# 4. Encode Categorical Variables
#-----
le = LabelEncoder()
for col in data.select_dtypes(include='object').columns:
    data[col] = le.fit_transform(data[col])

#-----
# 5. Final Structured Dataset
#-----
print("\nStructured Dataset Ready for HR Analytics:\n")
print(data.head())

print("\nDataset Summary:\n")
print(data.info())
```

Original Data:

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	\
0	1	Employee_1	Marketing	15	Master	53	
1	2	Employee_2	Operations	7	Bachelor	25	
2	3	Employee_3	IT	12	High School	51	
3	4	Employee_4	Operations	8	PhD	44	
4	5	Employee_5	Operations	15	Master	36	

	Gender	City	Monthly_Salary
0	Female	Delhi	111416
1	Female	Bangalore	95271
2	Female	Hyderabad	69064
3	Male	Delhi	95091
4	Female	Delhi	132450

Structured Dataset Ready for HR Analytics:

	EmployeeID	Name	Department	Experience_Years	Education_Level	Age	\
0	1	0	3	15	2	53	
1	2	11	4	7	0	25	
2	3	22	2	12	1	51	
3	4	33	4	8	3	44	
4	5	44	4	15	2	36	

	Gender	City	Monthly_Salary
0	0	2	111416
1	0	0	95271
2	0	3	69064
3	1	2	95091
4	0	2	132450

Dataset Summary:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 50 entries, 0 to 49

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	EmployeeID	50 non-null	int64
1	Name	50 non-null	int64
2	Department	50 non-null	int64
3	Experience_Years	50 non-null	int64
4	Education_Level	50 non-null	int64
5	Age	50 non-null	int64
6	Gender	50 non-null	int64
7	City	50 non-null	int64
8	Monthly_Salary	50 non-null	int64

dtypes: int64(9)

memory usage: 3.6 KB

None

/tmp/ipykernel_5943/1095290471.py:13: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use DataFrame.fillna(method='ffill', inplace=True) instead.

Start coding or [generate](#) with AI.

